

In-class assignment 2 (Individual): Java Coverage code / JaCoC0

AD/ SPRINT 3 Fall 2026

Note: The in-class assignments are compulsory for completing both the sprint implementation and the overall project. Failure to complete the in-class assignments will result in losing the implementation part of your sprint tasks and, consequently, your overall course grade. This will cause you to fail the course if your total points do not reach the required threshold of 2 points.

Tasks Objective

Continue the in-class assignment by generating and analyzing a code coverage report of the `TemperatureConverter` class.

Instructions

/ In-class assignments tasks, for those students who failed to do the assignment*

Step 1: Create the `TemperatureConverter` Class

- Create a new Java class named `TemperatureConverter`.
- Implement `fahrenheitToCelsius(double fahrenheit)`: returns $(fahrenheit - 32) * 5 / 9$
- Implement `celsiusToFahrenheit(double celsius)`: returns $(celsius * 9 / 5) + 32$
- Implement `isExtremeTemperature(double celsius)`: returns *true* if the temperature is below -40°C or above 50°C.

Step 2: Create the Test Class

- Right-click `TemperatureConverter` → Go To → Test → Create New Test.
- Name: `TemperatureConverterTest`
- Framework: JUnit5

Step 3: Write Unit Tests

- Initialize an instance of `TemperatureConverter`.
- Test `fahrenheitToCelsius` with multiple inputs.
- Test `celsiusToFahrenheit` with multiple inputs.
- Test `isExtremeTemperature` with edge cases

Step 4: Run the Tests

- Right-click `TemperatureConverterTest` → **Run**.

**/*

Step 1. Add JaCoCo Plugin to pom.xml

Sample code, you have to modify if it doesnot work with your JRE

```
<build>
  <plugins>
    <!-- JaCoCo Plugin for Code Coverage -->
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.11</version>
      <executions>
        <!-- Attach JaCoCo agent -->
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <!-- Generate coverage report -->
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Step 2: Ensure JUnit Dependency is Included

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
```

</dependencies>

Step 3. Run Maven Test with coverage

mvn clean test

Step 4: Generate HTML Coverage Report

After running the tests, JaCoCo automatically generates an HTML report.

Report Location:

target/site/jacoco/index.html

Step 5: Analyze Coverage

The HTML report provides:

- *Line Coverage (how many lines of code were executed)*
- *Branch Coverage (if all conditions were tested)*
- *Method Coverage*
- *Class Coverage*

Submission Instructions

1. Take a screenshot of your coverage code results.
2. Submit the screenshot to the Oma folder on the respected folder.
3. Share your public html folder which contain the coverage report (index.html)
4. Attach a link to your GitHub repository.